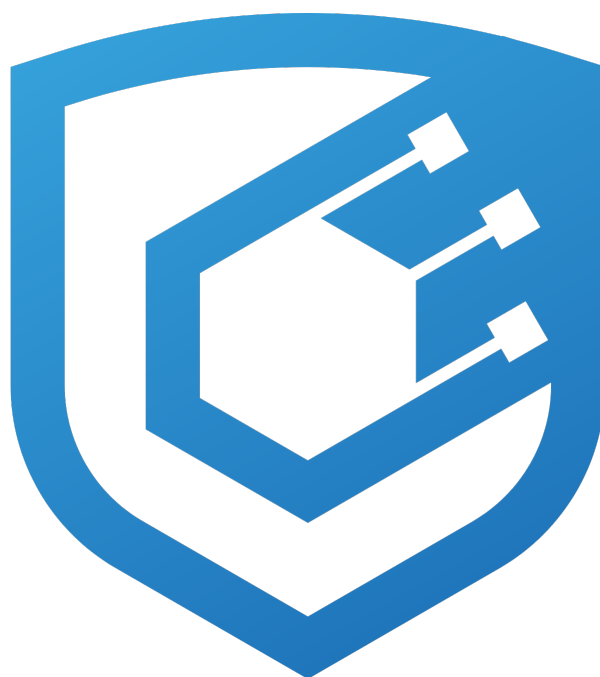




Securitize ZKsync Exit Bridge Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Immeas](#)

June 10, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
6.1	Centralization Risks	2
6.2	Defensive Improvements	3
7	Findings	5
7.1	Low Risk	5
7.1.1	SecuritizeBridge::executeVAAv1 never checks vm.consistencyLevel so non-finalized VAAs can be consumed, enabling reorg-based supply inflation	5
7.1.2	SecuritizeBridge::executeVAAv1 VAA payload omits destination-contract binding, enabling cross-instance double-mint across co-deployed bridge instances on one chain . . .	6
7.1.3	ZKSyncSecuritizeBridge::bridgeDSTokens hard-binds the Ethereum mint destination to msg.sender with no escape hatch for ZKSync smart-contract wallets	6
7.1.4	ZKSyncSecuritizeBridge::bridgeDSTokens enforces only a subset of the destination's is- suanance compliance, allowing burns that cannot be minted on Ethereum	7
7.2	Informational	9
7.2.1	Remove ZKSyncSecuritizeBridge::withdrawETH and in bridgeDSTokens revert when msg.value > 0	9
7.2.2	Remove unused import BaseDSContract from SecuritizeBridge.sol	9
7.3	Gas Optimization	10
7.3.1	Use assembly call for ETH transfers that discard return data	10
7.3.2	Redundant SLOAD of USDC in USDCBridgeV2::_transferUSDC	10
7.3.3	Redundant SLOAD of dsToken in SecuritizeBridge::bridgeDSTokensToAddress	10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

ZKSyncSecuritizeBridge is a source-only exit path that burns DS tokens on ZKSync Era and emits a DSToken-BridgeSend event, which an off-chain Kafka relay reads to mint the equivalent value on Ethereum through the Securitize Token Issuer.

5 Audit Scope

The audit scope was limited to:

```
contracts/bridge/ZKSyncSecuritizeBridge.sol
```

We also had another look at the previously audited files SecuritizeBridge.sol and USDCBridgeV2.sol.

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [ZKsync Exit Bridge](#) code provided by [Securitize](#). The findings consist of 4 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The bridge is a highly centralized, trust-heavy system by design; users must place substantial trust in the Securitize operator and the off-chain infrastructure when engaging with it.

6.2 Defensive Improvements

- Zero-amount and zero-recipient inputs to `USDCBridgeV2::sendUSDCCrossChainDeposit` produce opaque CCTP reverts instead of typed errors (`USDCBridgeV2.sol:206`); add entry-time `_amount > 0` and `_recipient != address(0)` checks, matching `SecuritizeBridge`.
- `SecuritizeBridge::executeVAAv1` mixes two address identities for EVM recipients: it decides whether the transfer is same-wallet or different-wallet by comparing the full 32-byte `sourceWallet` and `destinationAddress`, but it later mints to `address(uint160(uint256(destinationAddress)))`, which silently discards the upper 12 bytes (`SecuritizeBridge.sol:362,369`).

For canonical EVM addresses this is harmless because the upper 12 bytes are zero. For Solana-origin messages, however, the bridge can receive a user-supplied 32-byte `recipient` that is only checked as non-zero, so a non-canonical value can pass through and mint to the lower-20-byte EVM address. This does not produce a practical exploit today because Solana-sourced transfers take the different-wallet branch and the truncated EVM wallet must already be registered to the same investor, but the format mismatch should still be removed.

Validate the EVM destination format before truncating, and do it unconditionally on the EVM receive path rather than gating on `evmChains[sourceChain]` because the risky case is specifically a non-EVM source. Also apply the same canonical-format check before any source-side bridge publishes an explicit `bytes32` recipient to an EVM target chain, including Solana-side EVM-targeted recipients:

```
error NonCanonicalEVMAddress(bytes32 destinationAddress);

function _toCanonicalEvmAddress(bytes32 destinationAddress) private pure returns (address) {
    if (uint256(destinationAddress) >> 160 != 0) {
        revert NonCanonicalEVMAddress(destinationAddress);
    }
    return address(uint160(uint256(destinationAddress)));
}

// executeVAAv1: validate before any registry check or mint.
address destinationWallet = _toCanonicalEvmAddress(destinationAddress);

// bridgeDSTokensToAddress / _bridgeDSTokensInternal:
// only needed when the target chain expects an EVM recipient.
if (evmChains[_targetChain]) {
    _toCanonicalEvmAddress(_destinationAddress);
}
```

- The off-chain component (which is outside the scope of this audit) must be careful to not mint tokens on Ethereum immediately upon receiving a `DSTokenBridgeSend` event, but should wait for the ZKsync transaction to be in the [Verified](#) state.

Summary

Project Name	ZKsync Exit Bridge
Repository	bc-securitize-bridge-sc
Commit	011824a691c5...
Fix Commit	6ea08b492b95...
Audit Timeline	June 3rd - June 4th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	2
Gas Optimizations	3
Total Issues	9

Summary of Findings

[L-1] <code>SecuritizeBridge::executeVAAv1</code> never checks <code>vm.consistencyLevel</code> so non-finalized VAAs can be consumed, enabling reorg-based supply inflation	Acknowledged
[L-2] <code>SecuritizeBridge::executeVAAv1</code> VAA payload omits destination-contract binding, enabling cross-instance double-mint across co-deployed bridge instances on one chain	Acknowledged
[L-3] <code>ZKSyncSecuritizeBridge::bridgeDSTokens</code> hard-binds the Ethereum mint destination to <code>msg.sender</code> with no escape hatch for ZKSync smart-contract wallets	Resolved
[L-4] <code>ZKSyncSecuritizeBridge::bridgeDSTokens</code> enforces only a subset of the destination's issuance compliance, allowing burns that cannot be minted on Ethereum	Partially Resolved
[I-1] Remove <code>ZKSyncSecuritizeBridge::withdrawETH</code> and in <code>bridgeDSTokens</code> revert when <code>msg.value > 0</code>	Resolved
[I-2] Remove unused import <code>BaseDSContract</code> from <code>SecuritizeBridge.sol</code>	Resolved
[G-1] Use assembly <code>call</code> for ETH transfers that discard return data	Acknowledged
[G-2] Redundant SLOAD of USDC in <code>USDCBridgeV2::_transferUSDC</code>	Acknowledged
[G-3] Redundant SLOAD of <code>dsToken</code> in <code>SecuritizeBridge::bridgeDSTokensToAddress</code>	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 `SecuritizeBridge::executeVAAv1` never checks `vm.consistencyLevel` so non-finalized VAAs can be consumed, enabling reorg-based supply inflation

Description: The VAA signed by the Wormhole guardian network includes a `consistencyLevel` field that records the finality guarantee the source contract requested when it called `publishMessage`. On Ethereum the `consistencyLevel` is a guardian-interpreted sentinel, not a monotonic finality scale: 200 means instant (the guardians sign as soon as the transaction is observed, before block finality), 201 means "safe" (current default, per `contracts/bridge/SecuritizeBridge.sol:194`), and every other value - including values below 200 - is treated as finalized. `SecuritizeBridge::executeVAAv1` at `contracts/bridge/SecuritizeBridge.sol:328-400` verifies the VAA via `parseAndVerifyVM` (line 330), checks the emitter address (lines 338-340), validates replay protection (lines 358-359), and decodes the payload (lines 343-352) - but it never reads `vm.consistencyLevel`. The destination contract will accept a VAA at any consistency level, including instant.

The source-side `consistencyLevel` is owner-mutable via `updateConsistencyLevel` (`contracts/bridge/SecuritizeBridge.sol:237`) and is initialized to 201 at deployment (`contracts/bridge/SecuritizeBridge.sol:194`). If the owner sets it to 200 (instant), or if the current 201 "safe" level is insufficient for the source chain's finality guarantees in practice, guardians produce a valid, fully-verified VAA before the source burn transaction achieves finality. If the source chain subsequently reorgs and drops the burn transaction, the destination has already consumed the VAA and issued tokens - the investor's source-chain balance is restored while the destination mint stands. This violates the bridge's core conservation property that the total supply across all chains must not increase as a result of a bridge operation.

The in-scope contract documents the instant option explicitly at `contracts/bridge/SecuritizeBridge.sol:117` with the comment `ethereum: instant 200 - safe 201 - otherwise finalized`, confirming that a low consistency level is a supported configuration. No destination-side check closes the gap regardless of what level the source selects.

Files:

`SecuritizeBridge::executeVAAv1`

Impact: An investor can mint DS tokens on the destination chain without a finalized burn on the source chain. If the source transaction is reorganized away, the investor holds tokens on both chains for the same value, inflating total supply. The risk is proportional to both the probability of a source-chain reorg at the depth of the burn transaction and the consistency level configured by the owner. With `consistencyLevel = 200` (instant), a single block reorg is sufficient. With 201 ("safe"), the risk depends on the source chain's safe-head depth. Because `executeVAAv1` is permissionless, a VAA produced before finality can be delivered and consumed by anyone immediately after the guardians sign it.

Recommended Mitigation: In `SecuritizeBridge::executeVAAv1` for finalized-only delivery on Ethereum, reject the two non-final sentinels explicitly:

```
// 200 = instant, 201 = safe; every other value is finalized
if (vm.consistencyLevel == 200 || vm.consistencyLevel == 201) revert InsufficientConsistencyLevel();
```

For a safe-or-better policy, reject only instant:

```
if (vm.consistencyLevel == 200) revert InsufficientConsistencyLevel();
```

Because the acceptable classes can differ per source chain, prefer a per-source-chain allowlist (or enum) of accepted Wormhole finality classes rather than a single numeric comparison.

Securitize: Acknowledged; our intention is never to use instant finality in production/mainnet. We only set it to 200 in testing/testnet environments because otherwise we would need to wait around 20 minutes for every test run, which would significantly slow down development and CI execution.

So in practice instant finality is only ever used exclusively for testing purposes.

7.1.2 `SecuritizeBridge::executeVAAv1` VAA payload omits destination-contract binding, enabling cross-instance double-mint across co-deployed bridge instances on one chain

Description: The Wormhole VAA payload encodes `targetChain` (a chain-level Wormhole chain ID) but does not include the address of the destination `SecuritizeBridge` contract. `executeVAAv1` validates the source emitter (`emitterAddresses[sourceChain]`, `contracts/bridge/SecuritizeBridge.sol:338-340`), confirms `targetChain == _wormholeCore.chainId()` (`contracts/bridge/SecuritizeBridge.sol:355`), and guards against replay via `isVaaConsumed[vm.hash]` (`contracts/bridge/SecuritizeBridge.sol:358-359`) - but `isVaaConsumed` is per-proxy storage.

Nothing in the signed payload or in the verification logic binds a given VAA to one specific `SecuritizeBridge` instance. If two `SecuritizeBridge` proxies are deployed on the same destination chain - for example, a v2 deployment running alongside a legacy instance, or one bridge per DS token series - and both register the same source emitter via `setEmitterAddress`, a single source-chain burn produces one VAA that each instance independently accepts. Each proxy holds its own `isVaaConsumed` mapping, so the replay guard is satisfied independently on both, and `issueTokens` is called on each proxy's configured `dsToken` for the full bridged value.

Files:

`SecuritizeBridge::executeVAAv1, _encodePayload`

Impact: A single source burn can yield two destination issuances - one per co-deployed bridge instance sharing the same emitter configuration. The supply-conservation constraint that every investor must not receive more tokens on the destination than were burned on the source is broken for any deployment topology where two instances share an emitter on the same destination chain. The protocol currently provides no code-level defense against this topology, and `setEmitterAddress` does not enforce emitter uniqueness across bridge instances.

Recommended Mitigation: Bind the VAA to the specific destination bridge instance inside the guardian-signed payload. At the source, `_bridgeDSTokensInternal` already knows the configured destination via `bridgeAddresses[_targetChain]`; encode THAT value (not `address(this)`) as a destination-binding field in `SecuritizeBridge::_encodePayload` (`contracts/bridge/SecuritizeBridge.sol:503`), and in `executeVAAv1` decode it and require it equals `_addressToBytes32(address(this))` before consuming the replay slot.

Note that `address(this)` evaluated inside `_encodePayload` is the SOURCE bridge, not the destination, so encoding `address(this)` there would not bind the destination instance. A purely local extended replay key such as `(vm.hash, address(this))` is also insufficient, because the destination-contract identity must live inside the signed VAA body, not just in a local check.

Securitize: Acknowledged for now as this is a payload-breaking change and it is unlikely in practice for multiple `SecuritizeBridge` instances to be deployed on the same chain.

7.1.3 `ZKSyncSecuritizeBridge::bridgeDSTokens` hard-binds the Ethereum mint destination to `msg.sender` with no escape hatch for ZKSync smart-contract wallets

Description: `ZKSyncSecuritizeBridge::bridgeDSTokens` constructs both `sourceWallet` and `destinationAddress` from `msg.sender` via `_addressToBytes32(_msgSender())` (`contracts/bridge/ZKSyncSecuritizeBridge.sol:129`), enforcing `sourceWallet == destinationAddress` at the contract level. The companion function `bridgeDSTokensToAddress` is disabled with a `NotImplemented revert` (`contracts/bridge/ZKSyncSecuritizeBridge.sol:88`), eliminating any alternate-destination path.

ZKSync Era uses a different CREATE2 derivation for smart-contract wallets than Ethereum mainnet: a contract wallet address computed on ZKSync is not controlled by the same bytecode at the same address on Ethereum, unless the deployer has taken explicit steps to ensure address parity. This is more relevant on ZKSync than a typical chain because ZKSync has native account abstraction - smart-contract accounts are a first-class, common wallet type there, not an edge case.

An investor using a smart-contract wallet (a multisig, an account-abstraction wallet, or a proxy) on ZKSync that has not been identically reproduced on Ethereum cannot bridge their tokens: the minted tokens would go to an Ethereum address they do not control, and there is no code-level escape hatch to redirect the mint to an Ethereum address they do own.

Files:

ZKSyncSecuritizeBridge::bridgeDSTokens, bridgeDSTokensToAddress

Impact: Smart-contract wallet holders on ZKSync have no on-chain path to bridge their DS tokens to an Ethereum address they control. The tokens are burned on ZKSync and the off-chain Kafka relay would attempt to mint to an address they cannot sign for on Ethereum, effectively stranding the bridged value.

Recommended Mitigation: Enable an alternate-destination path on ZKSyncSecuritizeBridge::bridgeDSTokensToAddress (contracts/bridge/ZKSyncSecuritizeBridge.sol:88) subject to the same registration and compliance gate required by the Wormhole-based SecuritizeBridge different-wallet path.

Alternatively, document the smart-contract wallet limitation prominently so investors can verify address controllability before bridging. If the protocol wishes to continue prohibiting cross-wallet bridging, it should expose a pre-bridge address-parity check that reverts when the caller is a contract, allowing smart-contract wallet holders to learn of the limitation before their tokens are burned.

Securitize: Fixed in commit [162f19d](#)

Cyfrin: Verified. A msg.sender with code.size > 0 is rejected hence preventing smart wallets from bridging.

7.1.4 ZKSyncSecuritizeBridge::bridgeDSTokens enforces only a subset of the destination's issuance compliance, allowing burns that cannot be minted on Ethereum

Description: ZKSyncSecuritizeBridge::bridgeDSTokens (contracts/bridge/ZKSyncSecuritizeBridge.sol:113-131) burns DS tokens after checking only that the caller is a registered wallet and has sufficient *unlocked* balance. Within _validateLockedTokens (:135-152) region is read only to pick a lock period ((region == US) ? getUSLockPeriod() : getNonUSLockPeriod()); there is no region != FORBIDDEN check. The burn is also unconditional on eligibility, DSToken.burn to validateBurn to recordBurn (ComplianceServiceRegulated.sol:627) only adjusts counters, and getComplianceTransferableTokens (:781) computes only an unlocked-quantity.

The Ethereum mint, triggered by the off-chain relay through the existing Token Issuer, goes through the standard DSToken issuance path: validateIssuance (which is onlyToken, so it covers any issuance path) calls preIssuanceCheck and requires a zero code. That gate enforces a range of destination-side eligibility checks:

- destination-wallet registration (code 20)
- FORBIDDEN region (code 26)
- liquidate-only (code 90)
- force-accredited (codes 61/62)
- investor-count caps, reading *Ethereum's* counters (code 40)
- regional and global minimum holdings: minUSTokens / minEUTokens / minimumHoldingsPerInvestor (code 51)
- maximumHoldingsPerInvestor (code 52)
- the authorized-securities cap (enforced in validateIssuance)

On the documented relay path, TokenIssuer.issueTokens auto-registers an unregistered destination wallet before issuing, so the whitelist check (code 20) is satisfied there; code 20 only traps a raw DSToken.issueTokens to an unregistered wallet. Every destination condition outside "registered + unlocked" is therefore a burn-succeeds-but-mint-reverts trap, reachable via: a country reclassified to FORBIDDEN after onboarding; the ZKSync and Ethereum ComplianceConfigurationService instances being configured independently with divergent mappings; or the destination-side investor caps (the business-level cross-chain 100-investor limit is enforced per compliance-service instance, so the bridge cannot see Ethereum's counters).

Example: an investor whose country maps to FORBIDDEN on Ethereum calls bridgeDSTokens; the source checks pass and the tokens are burned; the relay submits the mint to the Token Issuer; preIssuanceCheck returns 26 and the mint reverts, the burned value is gone with no on-chain recovery. The same trap occurs on the holdings limits: e.g. a global minimumHoldingsPerInvestor of 1000 with a 100-token bridge to a fresh Ethereum wallet (code

51), or a bridge that brings the destination balance to or above `maximumHoldingsPerInvestor` (code 52). The Wormhole `SecuritizeBridge::_validateLockedTokens` (`SecuritizeBridge.sol:567`) shares the same source-side subset logic; on the ZKSync source-only bridge there is no on-chain destination contract, so a rejected mint can only be recovered off-chain.

Files:

`ZKSyncSecuritizeBridge::bridgeDSTokens, _validateLockedTokens` (`contracts/bridge/ZKSyncSecuritizeBridge.sol:131, 135-152`); also `SecuritizeBridge::_validateLockedTokens` (`contracts/bridge/SecuritizeBridge.sol:567`).

Impact: An investor whose destination issuance would be rejected can irreversibly burn tokens on ZKSync that cannot be minted on Ethereum, losing the burned value. The bridge has no on-chain recovery primitive, so remediation is operational. For `FORBIDDEN/force-accredited/liquidate-only` the block typically never clears (effectively permanent); for the investor-cap case the relay may mint later (value stuck rather than lost).

Recommended Mitigation: Add source-side pre-burn reverts for the eligibility conditions ZKSync can evaluate against its own services, so consistently-configured cases fail before any burn. All handles are reachable from the existing `dsServiceConsumer`:

```
uint256 internal constant FORBIDDEN = 4; // declare alongside the existing `US = 1`

// in _validateLockedTokens, after computing `region`:
if (region == FORBIDDEN) revert DestinationRestricted();

IDSLockManager lockManager =
↳ IDSLockManager(_dsServiceConsumer.getDSService(_dsServiceConsumer.LOCK_MANAGER()));
if (lockManager.isInvestorLiquidateOnly(_investorId)) revert InvestorLiquidateOnly();

if (
    (complianceConfigurationService.getForceAccredited() ||
     (region == US && complianceConfigurationService.getForceAccreditedUS())) &&
    !_registryService.isAccreditedInvestor(_msgSender())
) revert OnlyAccredited();
```

This relies on the ZKSync and Ethereum compliance state being in sync, it reads ZKSync's own config. It only covers the investor-property checks (region, liquidate-only, accreditation); the conditions that read Ethereum balances or counters (investor-count caps, min/max holdings, authorized-securities cap, destination-side registration) cannot be evaluated on the source.

Securitize: Tokens in multiple chains must be in sync, if an investor is from a forbidden region/country in zkSync and holds tokens then the burn actually is OK, because we are assuming here that Securitize is failing as a transfer agent and let a forbidden investor hold tokens. Same with accredited.

Liquidation mode is different I think, because we should stopped the burn in that case and revert. We can apply this validation. I think this is also valid for SecuritizeBridge contract, because destination bridge calls `issueTokens` and trigger same validation.

Fixed in commit [bfed319](#)

Cyfrin: Liquidate-only verified fixed. `FORBIDDEN-region` and `force-accredited` source checks not implemented.

7.2 Informational

7.2.1 Remove ZKSyncSecuritizeBridge::withdrawETH and in bridgeDSTokens revert when msg.value > 0

Description: ZKSyncSecuritizeBridge never uses ETH or msg.value but it needs the modifier payable on function bridgeDSTokens to satisfy compile-time interface requirements.

Recommended Mitigation: In ZKSyncSecuritizeBridge::bridgeDSTokens revert when msg.value > 0 and remove function withdrawETH.

Securitize: Fixed in commit [6ea08b4](#)

Cyfrin: Verified.

7.2.2 Remove unused import BaseDSContract from SecuritizeBridge.sol

Description: Remove unused import BaseDSContract from SecuritizeBridge.sol:

```
- import {BaseDSContract} from "@securitize/digital_securities/contracts/utils/BaseDSContract.sol";
```

Securitize: Fixed in commit [884b16e](#)

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 Use assembly call for ETH transfers that discard return data

Description: `(bool ok, ) = addr.call{value: amount}("")` copies the callee's entire return payload into memory even though it is discarded, wasting gas and exposing the caller to a return-bomb DoS. Use assembly call with zero output size instead.

```
contracts/bridge/ZKSyncSecuritizeBridge.sol
99:         (bool sent, ) = _to.call{value: amount}("");

contracts/bridge/SecuritizeBridge.sol
406:         (bool sent, ) = _to.call{value: amount}("");
491:         (bool ok, ) = payable(_msgSender()).call{value: extra}("");

contracts/bridge/USDCBridgeV2.sol
250:         (bool sent, ) = _to.call{value: amount}("");
```

Recommended Mitigation:

```
bool sent;
assembly {
    sent := call(gas(), _to, amount, 0, 0, 0, 0)
}
if (!sent) revert ETHTransferError();
```

Apply analogously to the refund at `SecuritizeBridge.sol:491`.

Securitize: Acknowledged.

7.3.2 Redundant SLOAD of USDC in `USDCBridgeV2::_transferUSDC`

Description: `USDCBridgeV2::_transferUSDC` reads the USDC state variable from storage twice - once as the `forceApprove` target and once as the `burnToken` argument. Its only caller `sendUSDCCrossChainDeposit` has already cached USDC into the local `_USDC` before invoking it. Passing the cached value as a parameter eliminates two extra SLOADs on this hot per-bridge path.

```
contracts/bridge/USDCBridgeV2.sol
211:         address _USDC = USDC;
219:         _transferUSDC(_amount, _targetChain, _recipient);
303:         IERC20(USDC).forceApprove(address(circleTokenMessenger), _amount);
312:         USDC,                                     // burnToken
```

Recommended Mitigation: Add an address `_usdc` parameter to `_transferUSDC`, pass `_USDC` from `sendUSDCCrossChainDeposit`, and use it at both lines 303 and 312. The line-302 comment ("Reading storage variables explicitly to avoid Yul errors") no longer applies once the value arrives as a parameter. Consider also caching `circleTokenMessenger` inside `_transferUSDC` since it is read twice at lines 303, 307.

Securitize: Acknowledged.

7.3.3 Redundant SLOAD of `dsToken` in `SecuritizeBridge::bridgeDSTokensToAddress`

Description: `SecuritizeBridge::bridgeDSTokensToAddress` re-reads `dsToken` from storage for the event argument `address(dsToken)`, but the internal `_bridgeDSTokensInternal` it just called already cached `dsToken` into the local `_dsToken` (line 427). This issues a fresh SLOAD on the user-facing hot path.

```
contracts/bridge/SecuritizeBridge.sol
300:         string memory investorId = _bridgeDSTokensInternal(_targetChain, _value,
↳ _destinationAddress);
301:         emit DSTokenBridgeSend(_targetChain, address(dsToken), _addressToBytes32(_msgSender()),
↳ _destinationAddress, investorId, _value);
```

```
427:         IDSToken _dsToken = dsToken;
```

Recommended Mitigation: Change `_bridgeDSTokensInternal` to return `(string memory investorId, address dsTokenAddr)`, set `dsTokenAddr = address(_dsToken)` inside it, and update the emit at line 301 to use the returned `dsTokenAddr` instead of `address(dsToken)`.

Securitize: Acknowledged.